

🧪 Traversal sequences stripped with superfluous URL-decode

PRACTITIONER

Tipo de explotación

Path Traversal al cargar una imagen del servidor.

La aplicación bloquea la entrada que contiene payload de Path Traversal. Luego realiza una decodificación de la URL de entrada antes de usarla.

Objetivo

Encontrar la vulnerabilidad Path Traversal de las rutas en la visualización de imágenes de productos.

Explicación técnica de la vulnerabilidad

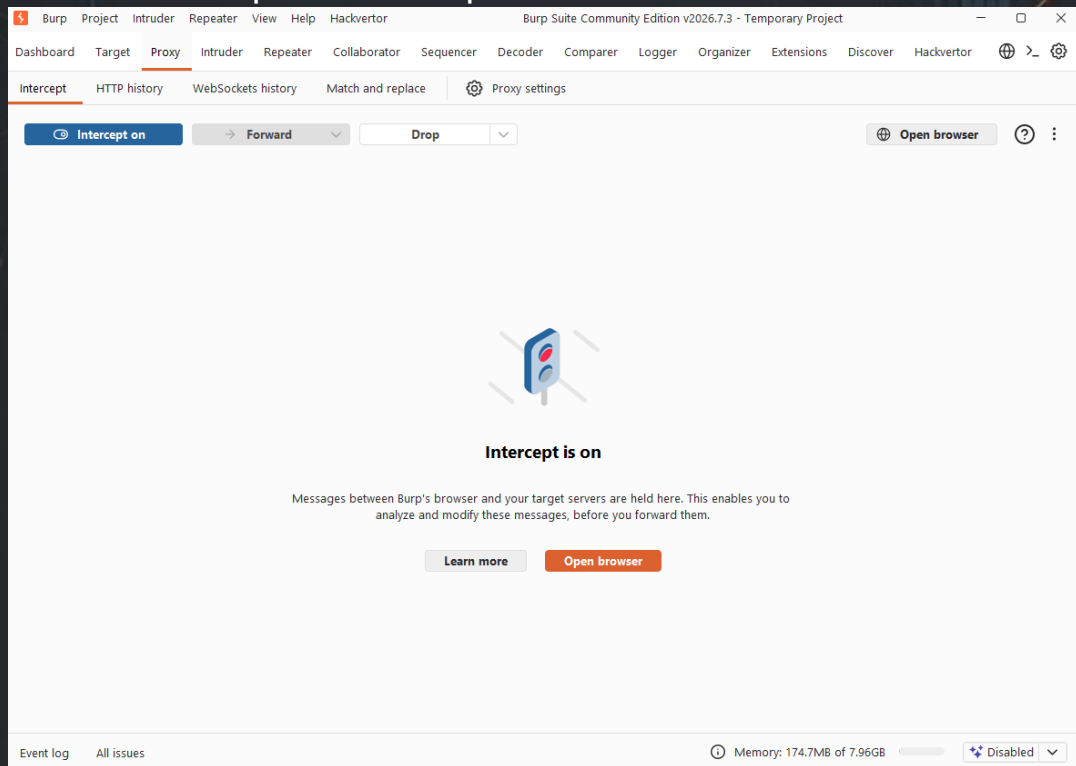
Estas vulnerabilidades permiten a un atacante leer archivos arbitrarios en el servidor que ejecuta una aplicación. Esto podría incluir:

- Código y datos de la aplicación.
- Credenciales de sistemas de back-end.
- Archivos sensibles del sistema operativo.

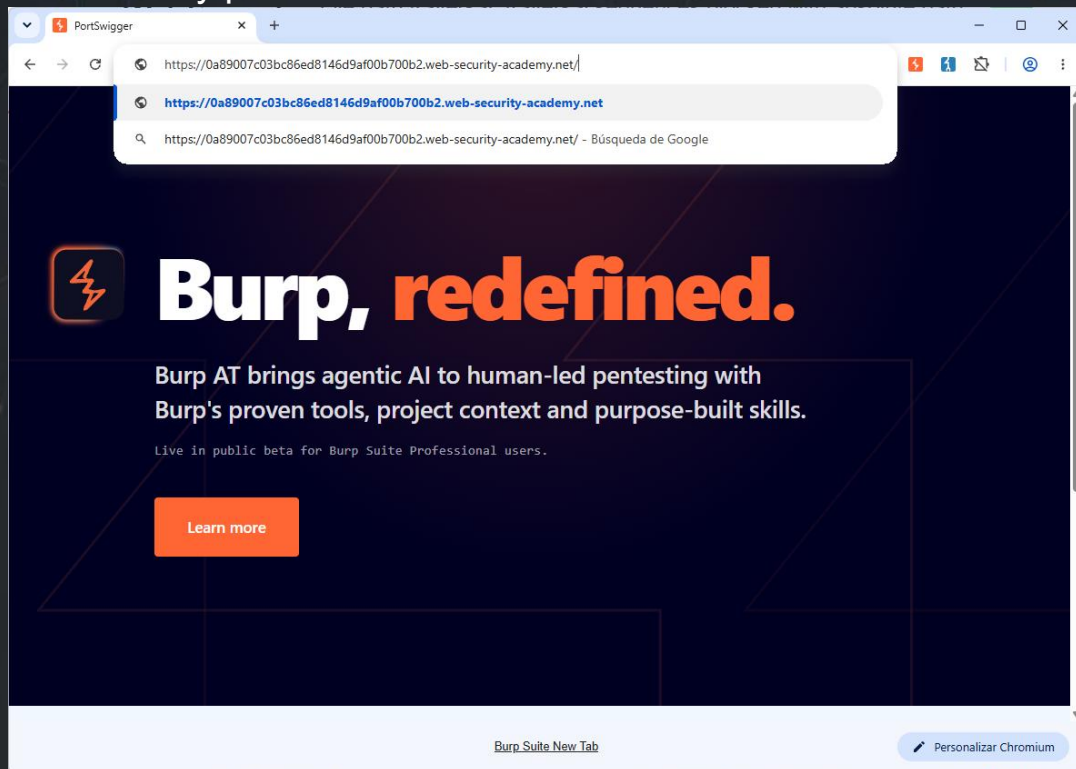
En algunos casos, un atacante podría incluso escribir en archivos arbitrarios en el servidor, lo que les permitiría modificar datos o el comportamiento de la aplicación y, finalmente, tomar el control total del servidor.

Procedimiento paso a paso

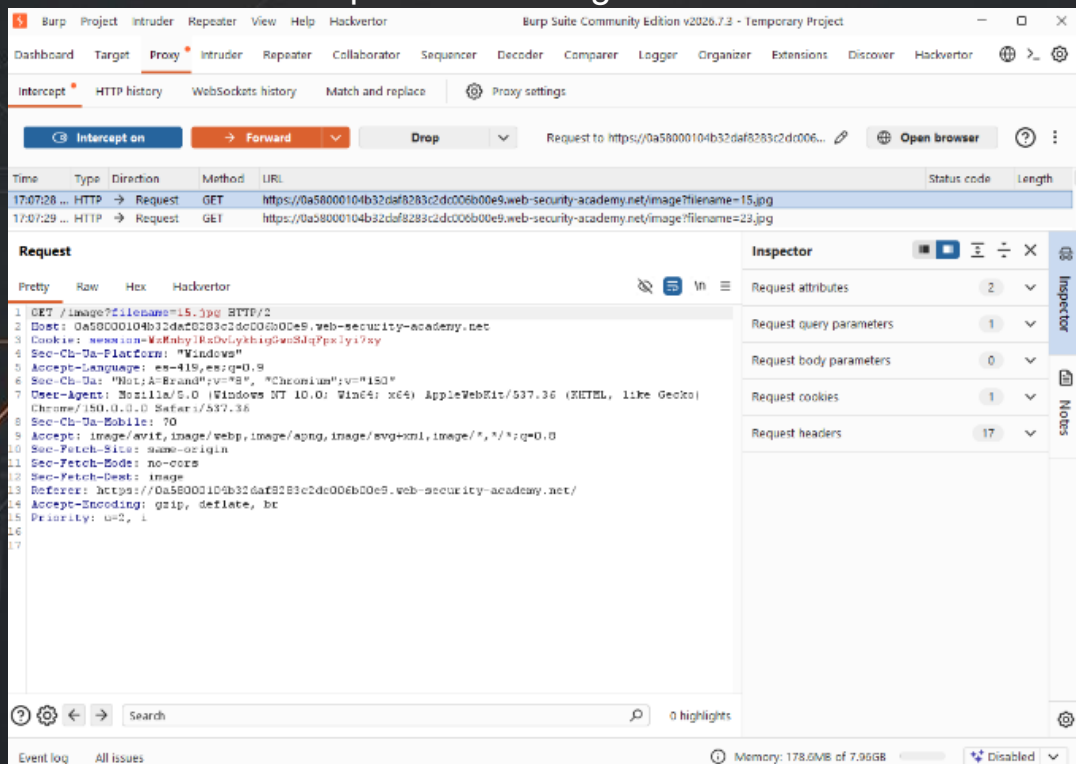
1. Abrir Burp Suite nos dirigimos a la pestaña Proxy.
2. Activamos la opción Intercept on.



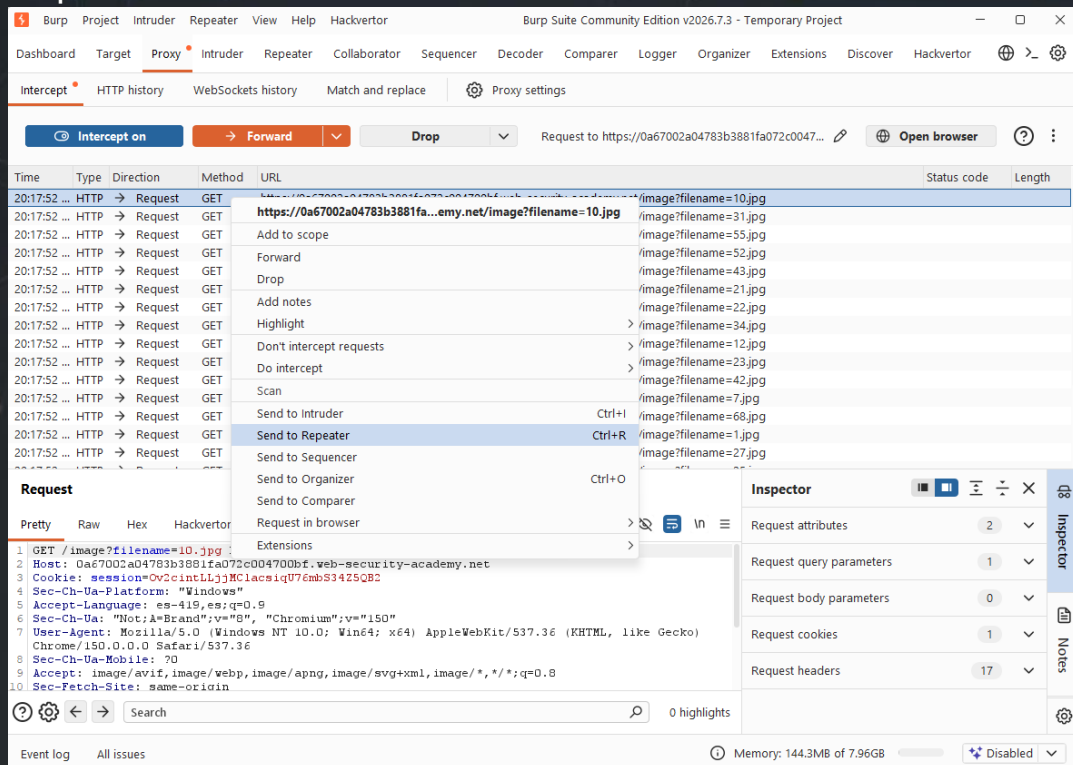
3. Dar clic en Open browser y pegar en este la dirección del laboratorio y presionar la tecla enter.



4. Regresar a Burp Suite y dar clic en Forward hasta que logremos ver la URL con el request de la imagen.



5. Dar clic con botón derecho al Request y dar clic en Sent to Repeater.



6. Dar clic en la pestaña Repeater.

7. En el apartado Reuquest logramos ver GET /image?filename=xxxx.jpg .

8. Borramos el nombre de la imagen e introducimos el Payload: %252e%252e%252f%252e%252e%252f%252e%252e%252fetc%252fpasswd .

Explicación del payload

En algunos entornos el servidor web elimina cualquier intento de payload de Path Traversal antes de entrar a la aplicación. A veces puedes eludir este tipo de restricción codificando el URL, o incluso codificando dos veces el URL, los caracteres `../`.

Dando como resultado `../` = `%2e%2e%2f` usando una sola codificación y `../` = `%252e%252e%252f` usando doble codificación, la doble codificación nos sirve ya que algunos servidores decodifican solo una vez los datos y comprueban que no estén los caracteres `../`.

Si codificamos todos los símbolos podríamos tener las siguientes opciones

	<code>../</code>	<code>../</code>	<code>../</code>	etc	<code>/</code>	passwd
Codificación	<code>%2e%2e%2f</code>	<code>%2e%2e%2f</code>	<code>%2e%2e%2f</code>	etc	<code>%2f</code>	passwd
Doble codificación	<code>%252e%252e%252f</code>	<code>%252e%252e%252f</code>	<code>%252e%252e%252f</code>	etc	<code>%252f</code>	passwd

Sin embargo, si la aplicación solo bloquea los caracteres juntos `../` también sería válido, por ejemplo: `..%252f..%252f..%252fetc/passwd`.

Mitigación recomendada

La forma más efectiva de prevenir path traversal es evitar pasar datos proporcionados por el usuario a las APIs del sistema de archivos por completo. Muchas funciones de la aplicación que hacen esto se pueden reescribir para ofrecer el mismo comportamiento de manera más segura.

Si no puedes evitar pasar datos del usuario a las APIs del sistema de archivos, recomendamos usar dos capas de defensa para prevenir ataques:

Valida la entrada del usuario antes de procesarla. Idealmente, compara la entrada con una lista blanca de valores permitidos. Si eso no es posible, verifica que la entrada contenga solo contenido permitido, como caracteres alfanuméricos.

Después de validar la entrada proporcionada, añade la entrada al directorio base y usa una API del sistema de archivos de la plataforma

para canonizar la ruta. Verifica que la ruta canonizada comience con el directorio base esperado.

Conclusiones

Al realizar los ejercicios me di cuenta lo fácil que es para un atacante realizar este ataque y la exposición de información vital para la administración del servidor, también aprendí la forma en que trabaja Burp Suite y conocer la amplia lista de laboratorios que nos ayudaran a encontrar vulnerabilidades explotables en nuestras aplicaciones web e información relevante que nos ayuda a mitigarlas.